

Database Design Document

Database Design and Implementation

Project: DevNexus MoMo Analytics

Team Members:

- Ajang Biar
- Chidiebele Anigbogu
- Sochukwuma Chukwu

1. Introduction

Project Overview

The DevNexus MoMo Analytics database is designed to manage and organize mobile money transaction data. The database stores information about users, transactions, transaction categories, transaction participants, and system logs.

The system is structured using a relational database model where related data is separated into different tables and connected using primary keys and foreign keys. This design supports transaction tracking, user management, categorization of transactions, and system monitoring.

2. Database Management System

The database implementation uses **MySQL** as the relational database management system.

The database is created under the name:

DevNexus_Momo_Analytics

The design uses relational tables with:

- Primary keys for unique record identification.
- Foreign keys for maintaining relationships between tables.
- ENUM fields for controlled values.
- Constraints to support data integrity.

3. Database Design Overview

The database consists of five main entities:

Entity	Description
USERS	Stores user account information including contact details and timestamps.
TRANSACTIONS	Stores mobile money transaction details including sender, receiver, amount, status, and transaction type.
TRANSACTION_CATEGORIES	Stores transaction classification information.
TRANSACTION_PARTICIPANTS	Records users involved in transactions and their roles.
SYSTEM_LOGS	Stores system activity logs related to transactions.

4. Entity Relationship Diagram (ERD)

The DevNexus MoMo Analytics ERD contains the following relationships:

USERS and TRANSACTIONS

A user can send multiple transactions and receive multiple transactions.

The relationship is implemented through:

- `sender_id`
- `receiver_id`

in the TRANSACTIONS table.

Both fields reference “users(user_id)”, this allows the database to identify the user initiating and receiving each transaction.

TRANSACTION_CATEGORIES and TRANSACTIONS

A transaction belongs to a transaction category.

The relationship is implemented through “category_id” in the TRANSACTIONS table.

The foreign key references “transaction_categories(category_id)” and this allows transactions to be grouped and analysed by category.

TRANSACTIONS and TRANSACTION_PARTICIPANTS

A transaction can involve multiple participants.

The TRANSACTION_PARTICIPANTS table acts as a linking table between transactions and users.

The relationship uses “transaction_id” and “user_id” as foreign keys.

The participant role is stored using:

- SENDER
- RECEIVER

TRANSACTIONS and SYSTEM_LOGS

Transactions can generate system logs.

The relationship is implemented through “transaction_id” in the SYSTEM_LOGS table. This allows system activities related to specific transactions to be stored and monitored.

5. ERD Design Rationale

Separation of Entities

The database separates users, transactions, categories, participants, and logs into individual tables. This prevents unnecessary duplication of information and improves data organisation.

For example, user details are stored once in the USERS table instead of being repeated for every transaction.

Primary Keys

Each table contains a primary key that uniquely identifies each record.

Primary keys include:

Table	Primary Key
USERS	user_id
TRANSACTIONS	transaction_id
TRANSACTION_CATEGORIES	category_id
TRANSACTION_PARTICIPANTS	participant_id
SYSTEM_LOGS	log_id

Foreign Keys

Foreign keys establish relationships between related tables.

Implemented foreign keys include:

Foreign Key	References
TRANSACTIONS.sender_id	USERS.user_id
TRANSACTIONS.receiver_id	USERS.user_id

TRANSACTIONS.category_id	TRANSACTION_CATEGORIES.category_id
TRANSACTION_PARTICIPANTS.transaction_id	TRANSACTIONS.transaction_id
TRANSACTION_PARTICIPANTS.user_id	USERS.user_id
SYSTEM_LOGS.transaction_id	TRANSACTIONS.transaction_id

6. Database Schema Documentation

USERS Table

The USERS table stores information about system users.

Column	Data Type	Description
user_id	INT	Unique user identifier
phone_number	VARCHAR(15)	User phone number
full_name	VARCHAR(100)	User full name
email	VARCHAR(100)	User email address
created_at	DATETIME	Record creation time
updated_at	DATETIME	Record update time

TRANSACTION_CATEGORIES Table

Stores transaction classification details.

Column	Data Type	Description
category_id	INT	Unique category identifier
category_name	VARCHAR(50)	Category name
description	VARCHAR(255)	Category description
created_at	DATETIME	Creation timestamp

TRANSACTIONS Table

Stores the main transaction information.

Column	Data Type	Description
transaction_id	INT	Unique transaction identifier
transaction_ref	VARCHAR(50)	Transaction reference
sender_id	INT	User sending transaction
receiver_id	INT	User receiving transaction
category_id	INT	Transaction category
amount	DECIMAL(15,2)	Transaction amount
transaction_type	ENUM	Transaction type
transaction_date	DATETIME	Transaction date
status	ENUM	Transaction status
balance_after	DECIMAL(15,2)	Balance after transaction
fee	DECIMAL(10,2)	Transaction fee
raw_xml	TEXT	Raw transaction XML data
created_at	DATETIME	Creation timestamp

TRANSACTION_PARTICIPANTS Table

Stores users involved in transactions.

Column	Data Type	Description
participant_id	INT	Unique participant identifier
transaction_id	INT	Related transaction
user_id	INT	Related user
role	ENUM	Participant role
created_at	DATETIME	Creation timestamp

SYSTEM_LOGS Table

Stores transaction-related system activity.

Column	Data Type	Description
log_id	INT	Unique log identifier
log_level	ENUM	Log severity level
log_message	TEXT	Log description
source_file	VARCHAR(255)	Source file
transaction_id	INT	Related transaction
logged_at	DATETIME	Log timestamp

7. SQL Implementation

The database was implemented using MySQL.

The SQL script includes:

- Database creation.
- Table creation.
- Primary key definitions.
- Foreign key relationships.
- ENUM constraints.
- Data type definitions.

The implemented database structure contains:

DevNexus_Momo_Analytics:

|— users

|— transactions

|— transaction_categories

|— transaction_participants

|— system_logs

8. CRUD Testing Documentation

8.1 CREATE — Tables Created

The following query verifies that all tables were created successfully.

SHOW TABLES;

Local instance 3306 - Warning - not supported

Administration Schemas Query 5 SQL File 3* SQL File 4*

SCHEMAS

Filter objects

- Enterprise Web Dev (3)
 - hello
 - motor_taxi
 - moto_taxi
 - sys

Object Info Session

Schema: DevNexus_Momo_Analytics

Query 5

```
1 • USE DevNexus_Momo_Analytics;  
2 • SHOW TABLES;
```

100% 13/2

Result Grid Filter Rows: Search Export:

Tables_in_devnexus_momo_analyt...

TABLE_NAME
SYSTEM_LOGS
TRANSACTION_CATEGORIES
TRANSACTION_PARTICIPANTS
TRANSACTIONS
users

Result 1

Action Output

Time	Action	Response	Duration / Fetch Time
22 22:35:46	SHOW TABLES	5 row(s) returned	0.0058 sec / 0.00001...

Query Completed

Result: All five tables are present.

8.2 READ – View All Users

Local instance 3306 (devnexus_momo_anal...) - Warning - not supported Local instance 3306 (DevNexus_Momo_Anal...) - Warning - not supported Local instance 3306 (DevNexus_Momo_Anal...) - Warning - not supported

Administration Schemas Query 1

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Dashboard
- Performance Reports
- Performance Schema Setup

Object Info Session

No object selected

Query 1

```
1 • SELECT * FROM transactions;  
2
```

100% 1/2

Result Grid Filter Rows: Search Edit: Export/Import:

transaction_...	transaction_...	sender_id	receiver_id	category_id	amount	transaction_ty...	transaction_date	status	balance_after	fee	raw_xml	created_at
1	TXN001	1	2	2	5000.00	TRANSFER	2026-09-15 16:30:00	COMPLETED	45000.00	50.00	<transaction><ref>TXN001</ref><amount>500...	2026-09-15 16:30:00
2	TXN002	2	3	1	2000.00	SEND	2026-09-15 16:35:00	COMPLETED	28000.00	20.00	<transaction><ref>TXN002</ref><amount>200...	2026-09-15 16:35:00
3	TXN003	3	4	3	7500.00	PAYMENT	2026-09-15 16:40:00	COMPLETED	12500.00	75.00	<transaction><ref>TXN003</ref><amount>750...	2026-09-15 16:40:00
4	TXN004	4	5	4	3500.00	PAYMENT	2026-09-15 16:45:00	PENDING	16500.00	35.00	<transaction><ref>TXN004</ref><amount>350...	2026-09-15 16:45:00
5	TXN005	5	1	2	10000.00	TRANSFER	2026-09-15 16:50:00	FAILED	30000.00	100.00	<transaction><ref>TXN005</ref><amount>100...	2026-09-15 16:50:00

transactions 1

Action Output

Time	Action	Response	Duration / Fetch Time
22 22:39:40	SELECT * FROM transactions LIMIT 0, 1000	5 row(s) returned	0.0022 sec / 0.00001...

Query Completed

Result: Five user records returned.

8.3 READ – View All Transactions

transaction...	transaction_...	sender_id	receiver_id	category_id	amount	transaction_ty...	transaction_date	status	balance_after	fee	raw_xml	created_at
1	TXN001	1	2	2	5000.00	TRANSFER	2026-09-15 16:30:00	COMPLETED	45000.00	50.00	<transaction><ref>TXN001</ref><amount>500...	2026-09-15 16:30:00
2	TXN002	2	3	1	2000.00	SEND	2026-09-15 16:35:00	COMPLETED	28000.00	20.00	<transaction><ref>TXN002</ref><amount>200...	2026-09-15 16:35:00
3	TXN003	3	4	3	7500.00	PAYMENT	2026-09-15 16:40:00	COMPLETED	12500.00	75.00	<transaction><ref>TXN003</ref><amount>750...	2026-09-15 16:40:00
4	TXN004	4	5	4	3500.00	PAYMENT	2026-09-15 16:45:00	PENDING	16500.00	35.00	<transaction><ref>TXN004</ref><amount>350...	2026-09-15 16:45:00
5	TXN005	5	1	2	10000.00	TRANSFER	2026-09-15 16:50:00	FAILED	30000.00	100.00	<transaction><ref>TXN005</ref><amount>100...	2026-09-15 16:50:00

Result: Five transaction records returned.

8.4 READ – JOIN Query

The following query joins transactions with sender, receiver, and category names.

SELECT t.transaction_ref, s.full_name AS sender, r.full_name AS receiver,
c.category_name, t.amount, t.status **FROM** transactions t **JOIN** users s **ON**
t.sender_id = s.user_id **JOIN** users r **ON** t.receiver_id = r.user_id **LEFT JOIN**
transaction_categories c **ON** t.category_id = c.category_id;

The screenshot shows a database management interface with a SQL query editor and a results grid. The query is as follows:

```

7  t.status
8  FROM transactions t
9  JOIN users s ON t.sender_id = s.user_id
10 JOIN users r ON t.receiver_id = r.user_id
11 LEFT JOIN transaction_categories c ON t.category_id = c.category_id;

```

The results grid displays the following data:

transaction_id	sender	receiver	category_name	amount	status
TXN001	Chidiebele Anigbogu	Emeka Obi	Money Transfer	5000.00	COMPLETED
TXN002	Emeka Obi	Adaobi Wale	Airtime	2000.00	COMPLETED
TXN003	Adaobi Wale	Funmi Wale	Bill Payment	7500.00	COMPLETED
TXN004	Funmi Wale	Adaobi oby	Merchant Payment	3500.00	PENDING
TXN005	Adaobi oby	Chidiebele Anigbogu	Money Transfer	10000.00	FAILED

Result: Each transaction shows the sender, receiver, and category correctly.

8.5 READ – Total Sent Per User

The following query calculates how much each user has sent.

```

SELECT u.full_name, COUNT(t.transaction_id) AS total_transactions, SUM(t.amount)
AS total_sent_rwf FROM users u JOIN transactions t ON t.sender_id = u.user_id
GROUP BY u.user_id, u.full_name ORDER BY total_sent_rwf DESC;

```

The screenshot shows a database management interface with a SQL query editor and a results grid. The query is as follows:

```

4 SUM(t.amount) AS total_sent_rwf
5 FROM users u
6 JOIN transactions t ON t.sender_id = u.user_id
7 GROUP BY u.user_id, u.full_name
8 ORDER BY total_sent_rwf DESC;

```

The results grid displays the following data:

full_name	total_transactio...	total_sent_r...
Adasobi obyi	1	10000.00
Adasobi Wale	1	7500.00
Chidebele Anigbogu	1	5000.00
Furmi Wale	1	3500.00
Emeka Obi	1	2000.00

The interface also shows a sidebar with navigation options like Administration, Schemas, and Query 1. The bottom status bar indicates the query was completed at 22:42:30, returning 5 rows in 0.0038 seconds.

Result: Each user's total sent amount is displayed.

8.6 READ – Volume Per Category

The following query calculates transaction volume per category.

```

SELECT tc.category_name, COUNT(t.transaction_id) AS txn_count, SUM(t.amount)
AS total_volume FROM transaction_categories tc LEFT JOIN transactions t ON
t.category_id = tc.category_id GROUP BY tc.category_id, tc.category_name;

```

The screenshot displays a database management interface with a SQL query and its results. The query is as follows:

```

3  COUNT(t.transaction_id) AS txn_count,
4  SUM(t.amount) AS total_volume
5  FROM transaction_categories tc
6  LEFT JOIN transactions t ON t.category_id = tc.category_id
7  GROUP BY tc.category_id, tc.category_name;

```

The results are shown in a table with the following data:

category_name	txn_count	total_volume
Airtime	1	2000.00
Money Transfer	2	15000.00
Bill Payment	1	7500.00
Merchant Payment	1	3500.00
Cash Withdrawal	0	0.00

The interface also shows a sidebar with navigation options like Administration, Schemas, Query 1, and a bottom status bar indicating the query was completed successfully.

Result: Each category shows its transaction count and total volume.

8.7 READ – Participants of a Transaction

The following query lists all participants for a specific transaction.

```
SELECT tp.role, u.full_name, u.phone_number, t.transaction_ref, t.amount, t.status
FROM transaction_participants tp JOIN users u ON tp.user_id = u.user_id JOIN
transactions t ON tp.transaction_id = t.transaction_id WHERE t.transaction_ref =
'TXN001';
```

The screenshot shows a database management tool interface. The query editor displays the following SQL query:

```
7 t.status
8 FROM transaction_participants tp
9 JOIN users u ON tp.user_id = u.user_id
10 JOIN transactions t ON tp.transaction_id = t.transaction_id
11 WHERE t.transaction_ref = 'TXN001';
```

The result grid shows the following data:

role	full_name	phone_number	transaction_ref	amount	status
SENDER	Chidiebele Anigbogu	08034509665	TXN001	5000.00	COMPLETED
RECEIVER	Emeka Obi	08034509645	TXN001	5000.00	COMPLETED

The interface also shows a sidebar with navigation options like Administration, Schemas, Query 1, and a bottom status bar indicating the query was completed successfully.

Result: The sender and receiver for TXN001 are shown.

8.8 READ – Error and Warning Logs

The following query retrieves error and warning logs.

SELECT sl.log_id, sl.log_level, sl.log_message, t.transaction_ref FROM system_logs
sl LEFT JOIN transactions t ON sl.transaction_id = t.transaction_id WHERE
sl.log_level IN ('ERROR', 'WARNING') ORDER BY sl.logged_at DESC;

The screenshot shows a database management interface with a query editor and a result grid. The query editor contains the following SQL code:

```
7 sl.logged_at
8 FROM system_logs sl
9 LEFT JOIN transactions t ON sl.transaction_id = t.transaction_id
10 WHERE sl.log_level IN ('ERROR', 'WARNING')
11 ORDER BY sl.logged_at DESC;
```

The result grid displays the following data:

log_id	log_level	log_message	source_file	transaction...	logged_at
4	ERROR	Transaction TXN005 failed	transaction_processor.py	TXN005	2026-09-15 16:50:00
3	WARNING	Transaction TXN004 is still pending	transaction_processor.py	TXN004	2026-09-15 16:45:00

The interface also shows a sidebar with navigation options like 'MANAGEMENT', 'INSTANCE', and 'PERFORMANCE'. The bottom status bar indicates 'Query Completed' and '2 row(s) returned'.

Result: Error and warning log entries are displayed.

8.9 UPDATE – Change User Email

The following update changes a user's email address.

UPDATE users SET email = 'chidi.newemail@alustudent.com' WHERE user_id = 1;

The screenshot displays a database management interface with a query editor and a result grid. The query editor contains the following SQL statements:

```
3 • UPDATE users
4   SET email = 'chidi.newemail@alustudent.com'
5   WHERE user_id = 1;
6
7 • SELECT user_id, full_name, email FROM users WHERE user_id = 1;
```

The result grid shows the output of the SELECT query, displaying one row of data:

user_id	full_name	email
1	Chidiebele Anigbogu	chidi.newemail@alustudent.com

The interface also includes a sidebar with navigation options (MANAGEMENT, INSTANCE, PERFORMANCE) and a bottom status bar indicating the query was completed successfully, returning 1 row(s) in 0.00076 seconds.

Result: The email was updated successfully.

8.10 DELETE — Remove a Participant

The following delete removes a participant record.

DELETE FROM TRANSACTION_PARTICIPANTS WHERE participant_id = 5;

The screenshot shows the SQL Server Enterprise Manager interface. The query editor displays the following SQL script:

```
1 • SELECT * FROM TRANSACTION_PARTICIPANTS;
2
3 • DELETE FROM TRANSACTION_PARTICIPANTS WHERE participant_id = 5;
4
5 • SELECT * FROM TRANSACTION_PARTICIPANTS;
```

The query execution results are shown in the 'Result Grid' tab. The grid displays the following data:

participant...	transaction...	user_id	role	created_at
1	1	1	SENDER	2026-09-15 16:30:00
2	1	2	RECEIVER	2026-09-15 16:30:00
3	2	2	SENDER	2026-09-15 16:35:00
4	2	3	RECEIVER	2026-09-15 16:35:00

The 'Action Output' tab shows the following information:

Action	Time	Action	Response	Duration / Fetch Time
3	22:46:07	SELECT * FROM TRANSACTION_PARTICIPANTS LIMIT 0, 1000	4 row(s) returned	0.00025 sec / 0.0000...

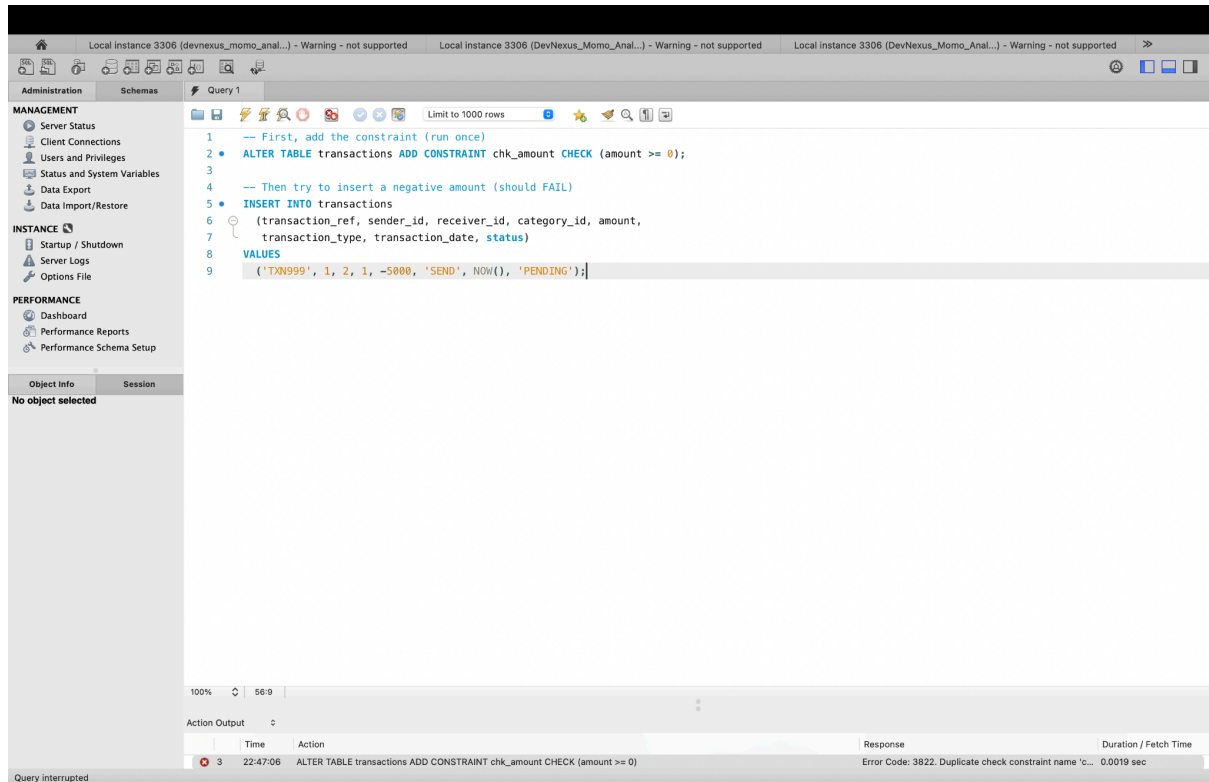
The status bar at the bottom indicates 'Query Completed'.

Result: The participant was removed successfully.

9.11 CHECK Constraint — Negative Amount Rejected

The following insert should fail because of the CHECK constraint.

```
INSERT INTO transactions (transaction_ref, sender_id, receiver_id, category_id,
amount, transaction_type, transaction_date, status) VALUES ('TXN999', 1, 2, 1, -5000,
'SEND', NOW(), 'PENDING');
```



Result: MySQL rejected the insert with a CHECK constraint error.

9.12 Trigger – Auto Logging

The following insert automatically creates a log entry.

INSERT INTO transactions (transaction_ref, sender_id, receiver_id, category_id, amount, transaction_type, transaction_date, status, balance_after, fee, created_at) VALUES ('TXN006', 1, 2, 1, 3000.00, 'SEND', '2026-09-15 17:30:00', 'COMPLETED', 42000.00, 30.00, NOW());

The screenshot shows a database management interface with a query editor and a results grid. The query editor contains the following SQL code:

```
19 '2026-09-15 17:30:00', 'COMPLETED', 42000.00, 30.00, NOW());
20
21 -- Verify the trigger created a log automatically
22 SELECT * FROM system_logs
23 WHERE transaction_id = (SELECT MAX(transaction_id) FROM transactions);
```

The results grid displays the output of the query, showing a single log entry in the system_logs table:

log_id	log_level	log_message	source_file	transaction...	trigger	logged_at
6	INFO	New transaction created: TXN006	trigger	6	6	2026-09-15 22:47:46

The interface also shows a sidebar with navigation options like Administration, Schemas, and Query 1. The bottom status bar indicates that the query was completed successfully, returning 1 row(s) in 0.0012 seconds.

Result: A new log entry appeared automatically in system_logs.

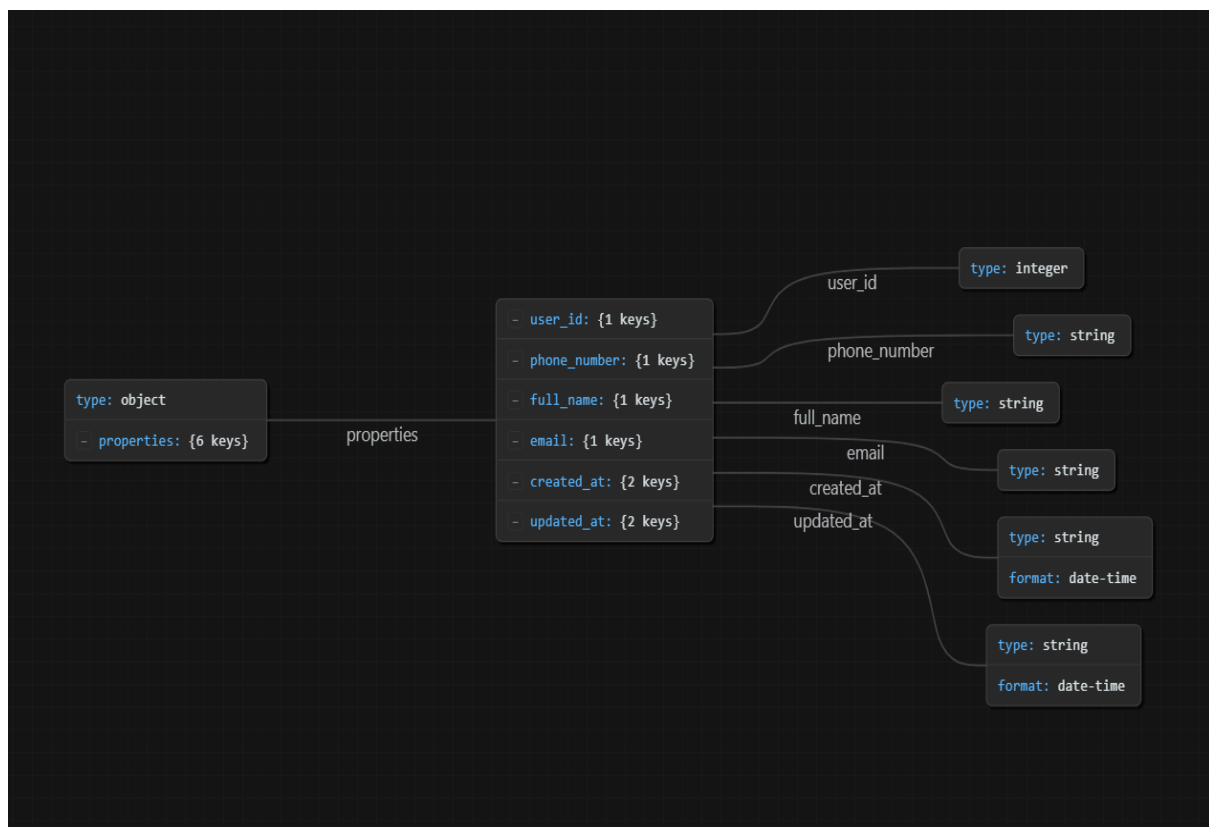
9. JSON Data Modelling

JSON schemas were created for the five entities in the DevNexus MoMo Analytics database. The schemas provide a JSON representation of the data stored in the relational database and define the expected data type for each field.

The JSON model follows the existing SQL structure. Each SQL table has a corresponding JSON object, and SQL `DATETIME` values are represented as JSON strings using the `date-time` format.

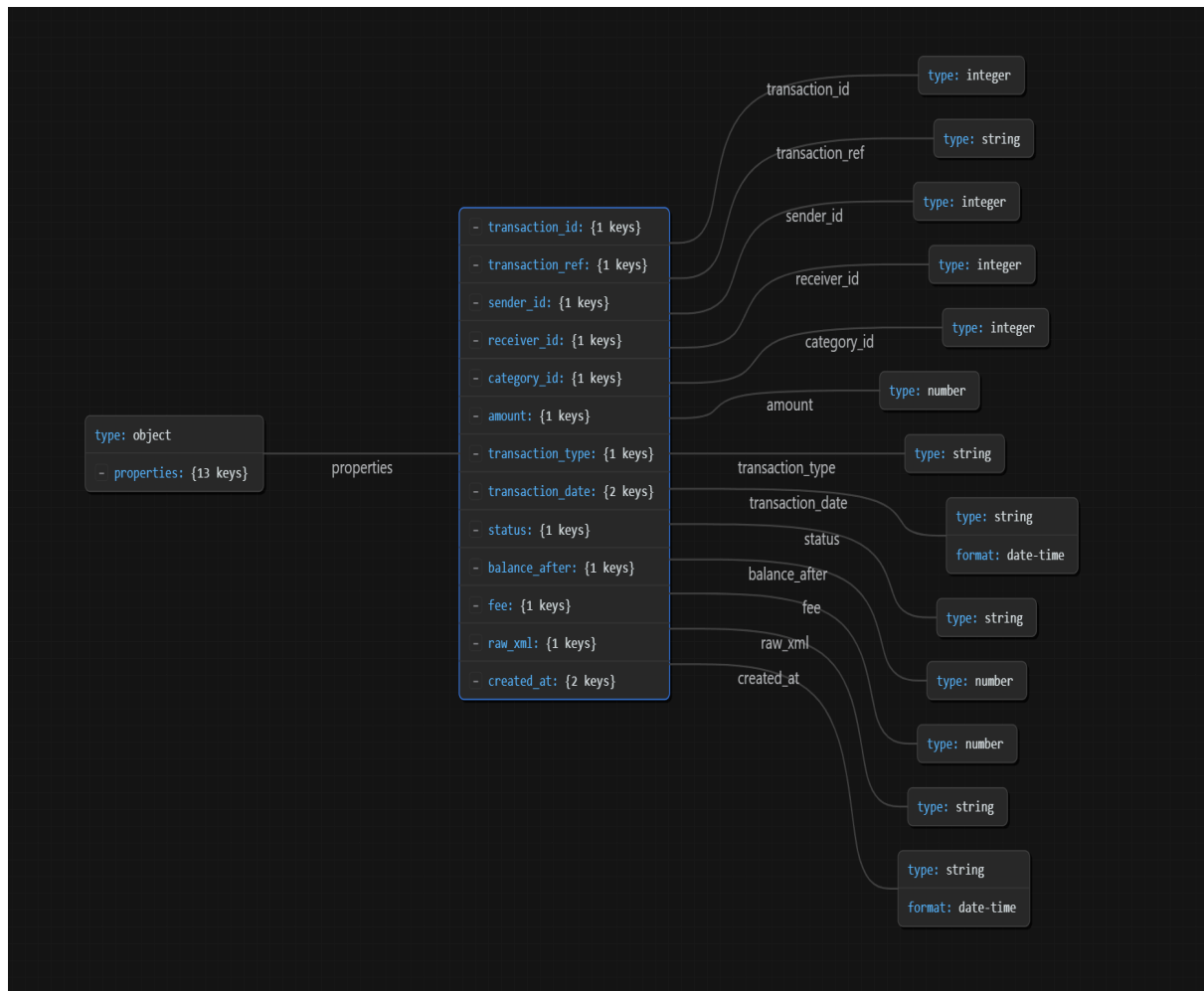
Users JSON Schema

The Users JSON schema represents records stored in the `users` table



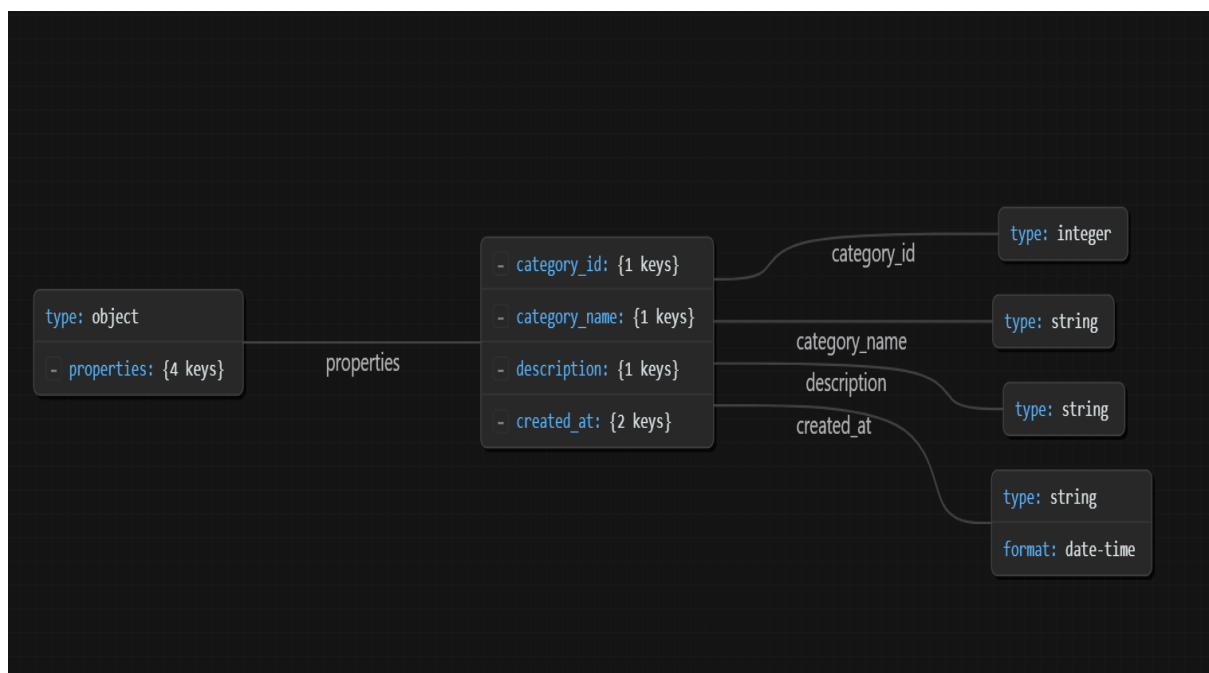
Transactions JSON Schema

The Transactions JSON schema represents records from the `transactions` table. It includes the transaction identifiers, sender and receiver references, category reference, financial values, transaction details, and timestamps.



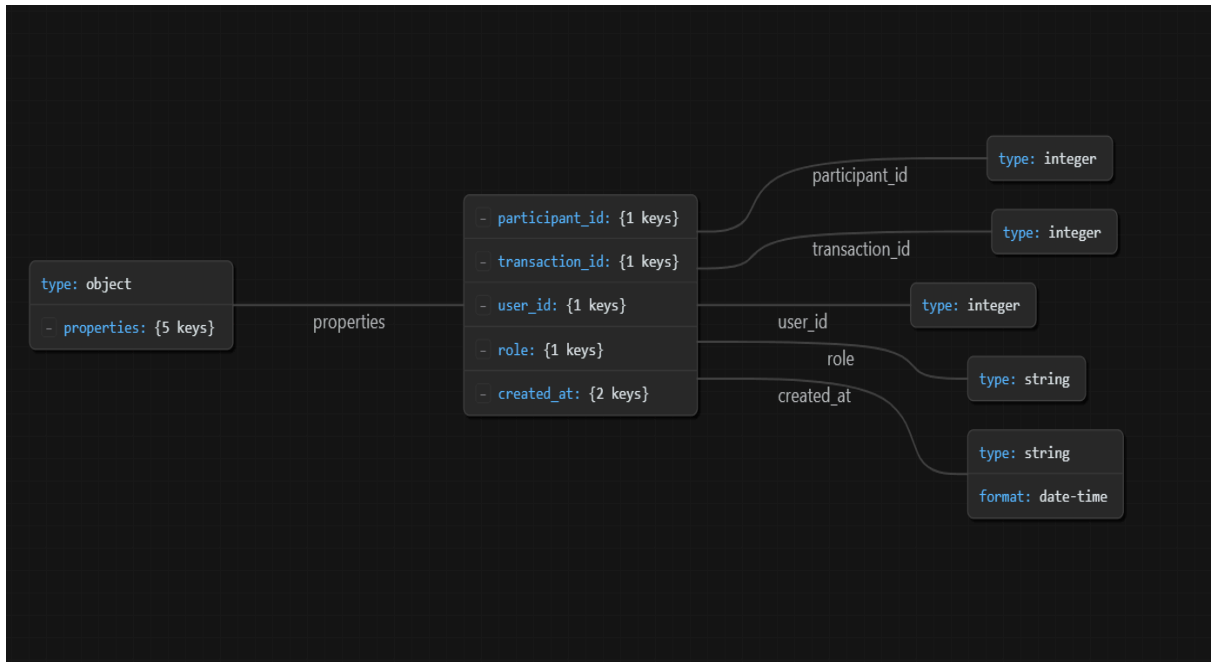
Transaction Categories JSON Schema

This schema represents records stored in `transaction_categories`.



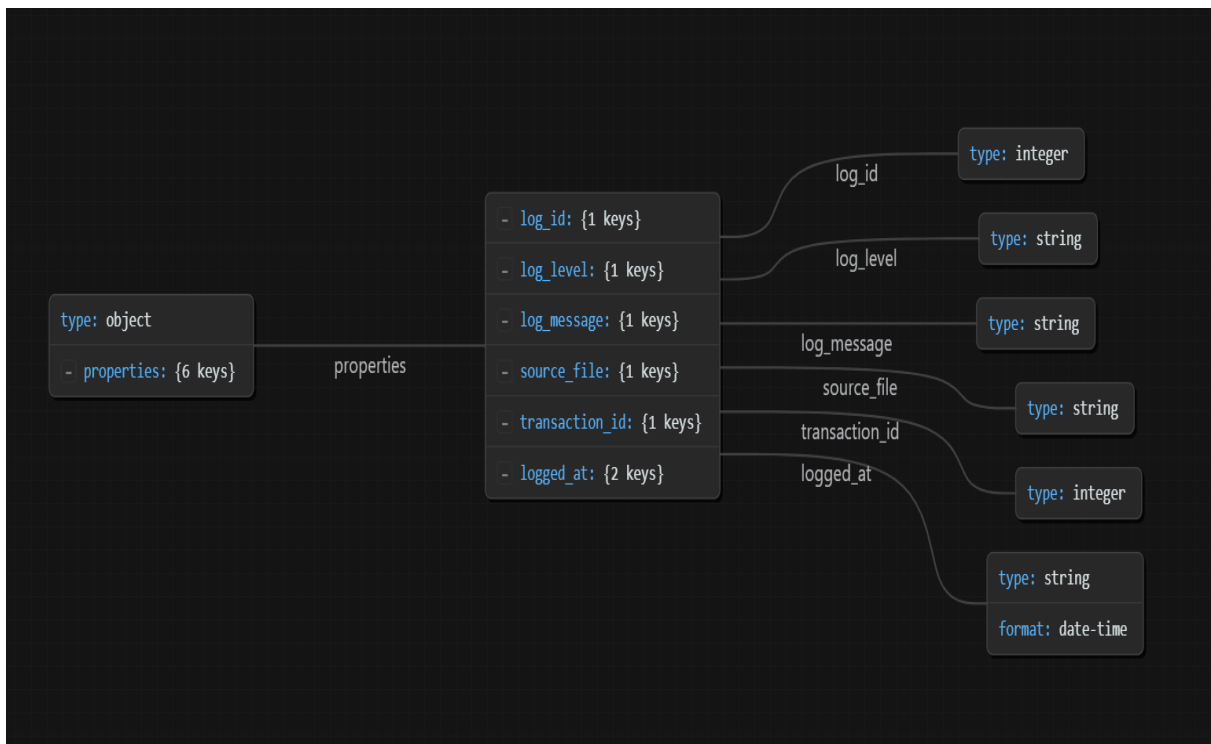
Transaction Participants JSON Schema

The Transaction Participants schema represents the relationship between a transaction and a participating user.



System Logs JSON Schema

The System Logs schema represents log records associated with transactions.



10. SQL-to-JSON Mapping

The JSON schemas map the fields of each MySQL table to equivalent JSON properties. The mapping keeps the same field names used in the relational database, making it straightforward to identify the source SQL column for each JSON property.

Data Type Mapping

MySQL Type Used	JSON Schema Type	Example
INT	integer	"user_id": 1
VARCHAR	string	"full_name": "Ajang Biar"
TEXT	string	"log_message": "..."
DECIMAL	number	"amount": 5000.00
ENUM	string	"status": "COMPLETED"
DATETIME	"string" with "date-time" format	"created_at": "2026-09-15T18:30:00"

Users Mapping

SQL Column	JSON Property	JSON Type
user_id	user_id	integer
phone_number	phone_number	string
full_name	full_name	string
email	email	string
created_at	created_at	string, date-time
updated_at	updated_at	string, date-time

Transactions Mapping

SQL Column	JSON Property	JSON Type
transaction_id	transaction_id	integer
transaction_ref	transaction_ref	string
sender_id	sender_id	integer
receiver_id	receiver_id	integer
category_id	category_id	integer
amount	amount	number
transaction_type	transaction_type	string
transaction_date	transaction_date	string, date-time
status	status	string
balance_after	balance_after	number
fee	fee	number
raw_xml	raw_xml	string
created_at	created_at	string, date-time

The [sender_id](#), [receiver_id](#), and [category_id](#) properties preserve the foreign key values used by the SQL implementation. They are not converted into nested JSON objects in the schemas provided.

Transaction Categories Mapping

SQL Column	JSON Property	JSON Type
category_id	category_id	integer
category_name	category_name	string
description	description	string
created_at	created_at	string, date-time

Transaction Participants Mapping

SQL Column	JSON Property	JSON Type
participant_id	participant_id	integer
transaction_id	transaction_id	integer
user_id	user_id	integer
role	role	string
created_at	created_at	string, date-time

The SQL foreign keys `transaction_id` and `user_id` remain integer identifier properties in JSON.

System Logs Mapping

SQL Column	JSON Property	JSON Type
log_id	log_id	integer
log_level	log_level	string
log_message	log_message	string
source_file	source_file	string
transaction_id	transaction_id	integer
logged_at	logged_at	string, date-time

The `transaction_id` property maintains the reference between the system log and its associated transaction.

10.7 SQL and JSON Constraint Differences

The SQL implementation contains some constraints that are not currently represented by the JSON schemas.

For example, the SQL definition restricts `transaction_type` to:

- SEND
- RECEIVE
- TRANSFER
- PAYMENT

The SQL definition restricts `status` to:

- PENDING
- COMPLETED
- FAILED
- CANCELLED

The SQL definition restricts participant `role` to:

- SENDER
- RECEIVER

The SQL definition also restricts `log_level` to:

- INFO
- WARNING
- ERROR
- DEBUG

The current JSON schemas define these fields only as `"type": "string"`. They therefore describe the data type but do not currently enforce the same allowed values as the MySQL `ENUM` definitions.

Also, none of the five supplied schemas currently contains a `required` array. Therefore, the schemas do not currently require any property to be present, even where the SQL schema uses `NOT NULL`.

That distinction is worth documenting because it accurately reflects what your team has actually implemented rather than claiming that the JSON validation is stricter than it is.

11. Conclusion

The DevNexus MoMo Analytics database was designed to organize and manage mobile money transaction data in a clear manner. The database separates important information into different tables such as users, transactions, transaction categories, transaction participants, and system logs.

Using MySQL, the database structure was implemented with primary keys and foreign keys to maintain relationships between the tables and keep the data consistent. The JSON schemas and SQL-to-JSON mapping also provide a way to represent the database information in a structured format.

Overall, the design provides a good foundation for storing and managing transaction records. The documentation covers the main parts of the database design, including the ERD, SQL implementation, and JSON representation.